

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет: Компьютерных систем и сетей

Кафедра: Электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЁТ
к лабораторной работе № 1
на тему
«КАЛЬКУЛЯТОР НАСЕЛЕНИЯ МИРА»
Вариант 7

Студент:

А.И. Баяк

Преподаватель:

Д.В. Куприянова

Минск 2026

1 ЦЕЛЬ И ЗАДАЧИ

Цель работы – изучить основные возможности интегрированной среды разработки IntelliJ IDEA при создании объектно-ориентированного приложения на языке Java, освоить запуск и отладку программы с графическим интерфейсом, а также применить архитектурный паттерн MVC с активной моделью.

В соответствии с вариантом 7 необходимо разработать утилиту для расчета численности населения. Для выполнения работы поставлены следующие задачи:

1. Создать и настроить проект Java в IntelliJ IDEA, изучить средства редактирования, сборки, запуска и отладки;
2. Подобрать средства IntelliJ IDEA для работы с UML и SysML v2 и отразить результат настройки в отчёте;
3. Реализовать проверку входных данных и вывод понятного сообщения при ошибке;
4. Разделить приложение на модель, представление и контроллер и реализовать активное уведомление представления моделью;
5. Проверить основные сценарии работы программы и подготовить ответы на контрольные вопросы.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЕ

2.1 Интегрированная среда разработки и её компоненты

Интегрированная среда разработки объединяет редактор исходного кода, средства навигации и рефакторинга, систему сборки, запуск приложения, отладчик, терминал и интеграцию с системой контроля версий. IntelliJ IDEA анализирует Java-код во время редактирования, показывает ошибки и предупреждения, предлагает быстрые исправления и позволяет запускать класс с методом main непосредственно из редактора.

2.2 Редактирование и навигация в IntelliJ IDEA

Редактор IntelliJ IDEA поддерживает автодополнение, импорт классов, форматирование, безопасное переименование и поиск использований. Для перехода к объявлению используется Ctrl+B, для поиска использований – Alt+F7, для форматирования – Ctrl+Alt+L, а для вызова намерений и быстрых исправлений – Alt+Enter. Структура проекта отображается в окне Project, сообщения компилятора и программы – в окнах Build и Run.

2.3 Сборка, запуск и отладка Java-приложения

Исходные файлы компилируются компилятором javac в байт-код JVM. В IntelliJ IDEA проект собирается командой Build Project, а конфигурация запуска класса App выполняется командами Run и Debug. Точка останова устанавливается щелчком в поле слева от строки или сочетанием Ctrl+F8. Во время остановки окно Debug показывает стек вызовов и значения переменных. Команды Step Over, Step Into и Step Out позволяют выполнять программу пошагово.

2.4 Паттерн MVC

MVC разделяет приложение на три части. Model хранит состояние и выполняет предметную логику. View формирует интерфейс и отображает данные. Controller обрабатывает действия пользователя и передаёт команды модели. Такое разделение уменьшает связанность классов и позволяет проверять декодирование независимо от Swing-интерфейса.

2.5 Активная и пассивная модель

В варианте с пассивной моделью контроллер после изменения данных сам инициирует обновление представления. Активная модель сообщает подписчикам об изменении состояния. В разработанном приложении PopulationModel использует паттерн «Наблюдатель» (Observer) через внутренний интерфейс ModelListener.

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Постановка задачи

Разработано приложение для оценки численности населения мира по указанной дате. Пользователь нажимает кнопку «Ввести данные» на главной форме — открывается отдельное диалоговое окно, где задаются день, месяц и год. За расчёт отвечает экспоненциальная модель роста с базовой точкой в 2022 году (7,95 млрд). Диапазон допустимых лет — от 1900 до 2200.

3.2 Настройка среды и расширений для UML и SysML v2

В IntelliJ IDEA открыт проект с каталогом src/main/java и пакетом com.example. В качестве Project SDK выбран JDK 25. Класс com.example.Main содержит метод main и используется в конфигурации запуска. Для UML применяется плагин plantuml4idea и файл docs/diagram.puml. Для описания структуры системы применена текстовая нотация SysML v2. Средства IntelliJ IDEA для работы с UML и SysML v2 продемонстрированы на рисунке 3.1.

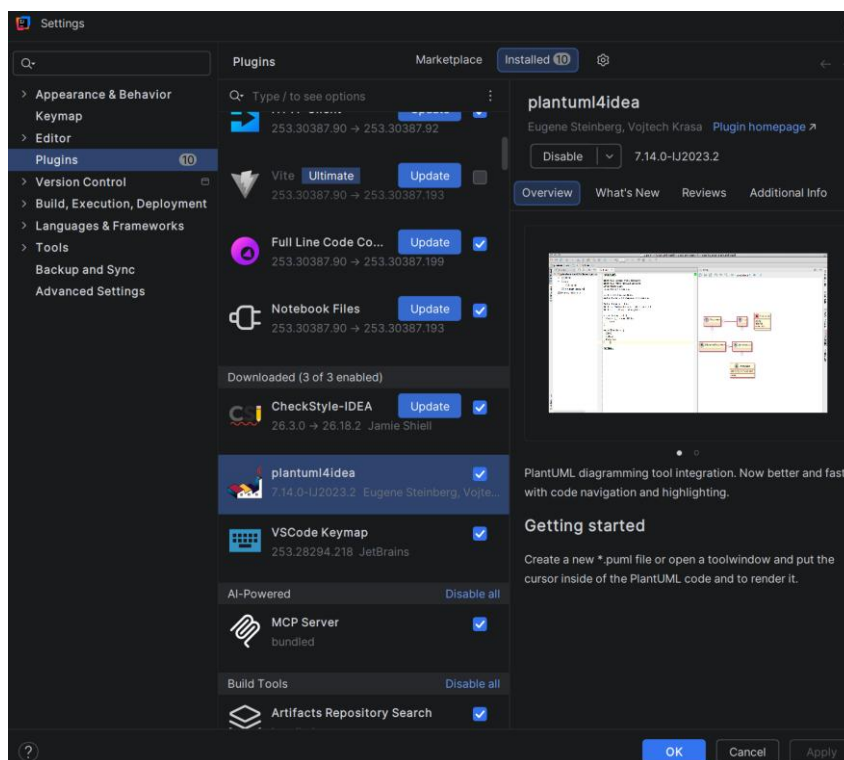


Рисунок 3.1 – Средства IntelliJ IDEA для работы с UML и SysML v2

Структурное представление системы в нотации, близкой к SysML v2:

```
package PopulationCalculator {  
    part def PopulationModel;  
    part def MainView;  
    part def InputDialog;
```

```

part def PopulationController;

part application {
    part model : PopulationModel;
    part view : MainView;
    part dialog : InputDialog;
    part controller : PopulationController;
    connect view to controller;
    connect controller to model;
    connect model to view;
}
}

```

3.3 Демонстрация отладчика

Работа отладчика демонстрируется в методе `calculatePopulation` класса `PopulationModel`. Точка останова устанавливается на строке вычисления `calculatedPopulation` (рисунок 3.2). При остановке в окне `Debug` можно наблюдать переменную `daysBetween`, хранящую число дней между базовой датой 01.01.2022 (`START_YEAR`) и введенной пользователем, переменную `elapsedYears` — число прошедших лет, и переменную `calculatedPopulation` — итоговое значение численности населения. С помощью функции `Evaluate Expression` (`Alt+F8`) можно вычислить промежуточное выражение `Math.exp(ANNUAL_GROWTH_RATE * elapsedYears)` и убедиться в корректности расчёта; пошаговое выполнение (`Step Over`) позволяет проследить, как изменение входной даты влияет на итоговый результат. Остановка программы в отладчике IntelliJ IDEA продемонстрирована на рисунке 3.2.

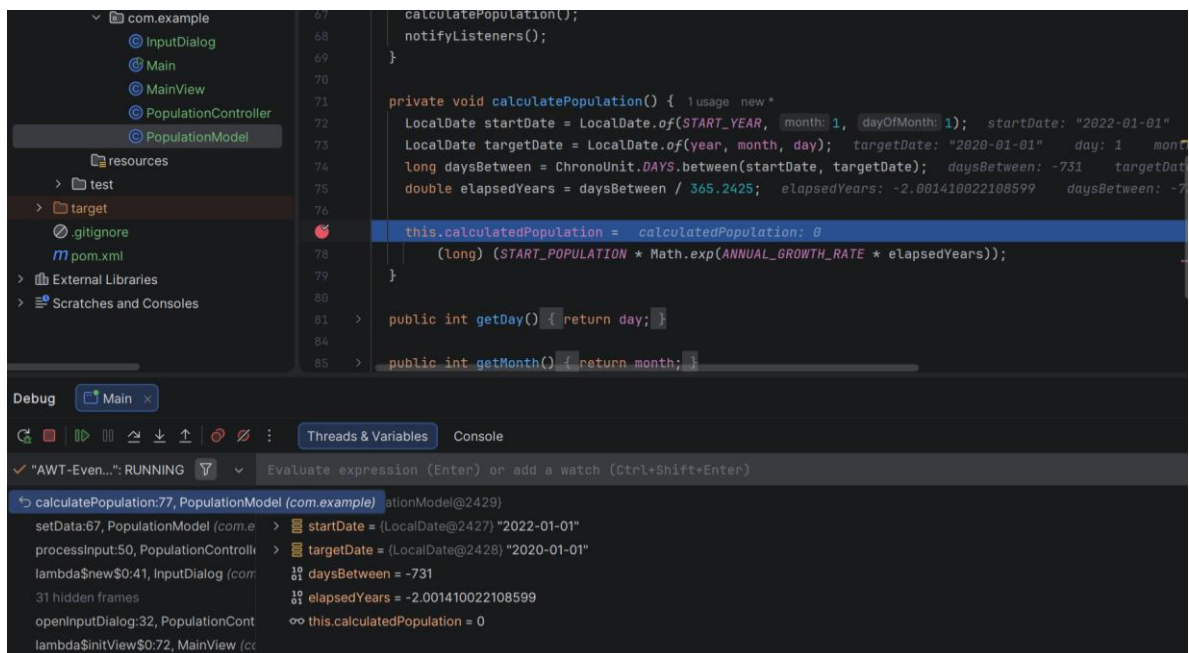


Рисунок 3.2 – Остановка программы в отладчике IntelliJ IDEA

3.4 Структура проекта и каркас MVC с активной моделью

- Проект разделён на следующие компоненты:
- PopulationModel – хранит дату и результат расчета, содержит бизнес-логику вычисления населения и механизм уведомления слушателей через вложенный интерфейс ModelListener;
 - MainView – главное окно приложения, реализует PopulationModel.ModelListener для автоматического обновления при изменении данных;
 - InputDialog – модальное окно для ввода даты; получает данные из модели через метод setValues(...) перед показом;
 - PopulationController – обрабатывает действия пользователя, выполняет парсинг строк в числа, передаёт данные в модель;
 - Main – точка входа, инициализирующая компоненты.

Диаграмма типов и интерфейсов приложения представлена на рисунке 3.3

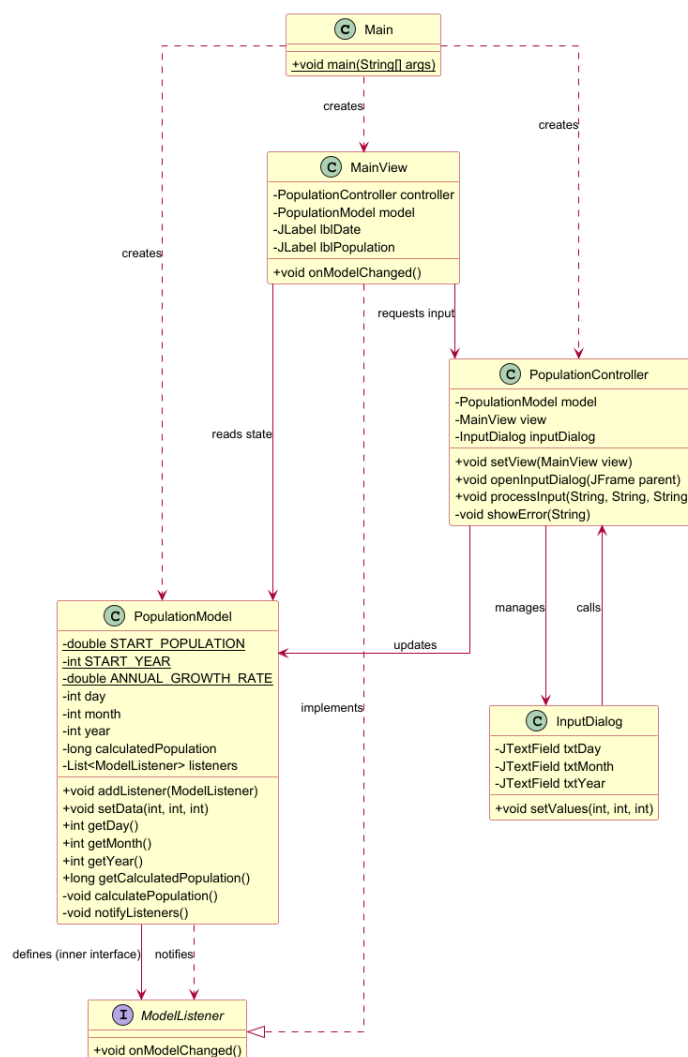


Рисунок 3.3 – Диаграмма типов и интерфейсов приложения

Взаимодействие компонентов построено по паттерну MVC с активной моделью: контроллер обрабатывает ввод пользователя и передаёт данные в модель, а модель после изменения состояния сама уведомляет подписанное представление через механизм слушателей.

3.5 Модель

PopulationModel реализует активную модель. Метод setData выполняет многоуровневую проверку: проверку диапазонов (день 1–31, месяц 1–12, год 1900–2200) и проверку фактического существования даты через LocalDate.of(). После валидации вызывается calculatePopulation(), которая вычисляет численность населения по экспоненциальной формуле роста. Базовая точка — январь 2022 года (население 7,95 млрд человек), среднегодовой темп прироста — 0,98%. Число прошедших лет определяется через ChronoUnit.DAYS.between() — это позволяет точно учесть високосные годы. После пересчёта модель уведомляет подписчиков через notifyListeners()..

3.6 Представление

MainView отображает текущую дату и рассчитанную численность населения. Благодаря реализации интерфейса ModelListener, метод onModelChanged() обновляет текстовые метки мгновенно после того, как модель зафиксировала изменения, — при этом View читает актуальные данные через геттеры модели (getDay(), getMonth(), getYear(), getCalculatedPopulation()). InputDialog обеспечивает ввод даты с поддержкой клавиши Enter в каждом поле для ускорения работы.

3.7 Контроллер

PopulationController управляет жизненным циклом InputDialog и отвечает за безопасный перевод пользовательского ввода из текстового формата в числовой. В случае NumberFormatException или IllegalArgumentException контроллер перехватывает ошибку и выводит её пользователю через JOptionPane.

3.8 Запоминание данных и обработка ошибок

Приложение предотвращает ввод некорректных дат. Сообщения об ошибках (например, «Указанной даты не существует.») формируются в модели и передаются через контроллер во View. Восстановление данных реализовано через метод inputDialog.setValues(), который вызывается перед показом окна.

4 РЕЗУЛЬТАТЫ РАБОТЫ

В IntelliJ IDEA создано и проверено Java-приложение варианта 7. Программа вычисляет численность населения, сохраняет последнее успешное состояние и сообщает о некорректном вводе. Архитектура соответствует MVC с активной моделью. Работа приложения продемонстрирована на рисунках 4.1, 4.2, 4.3.

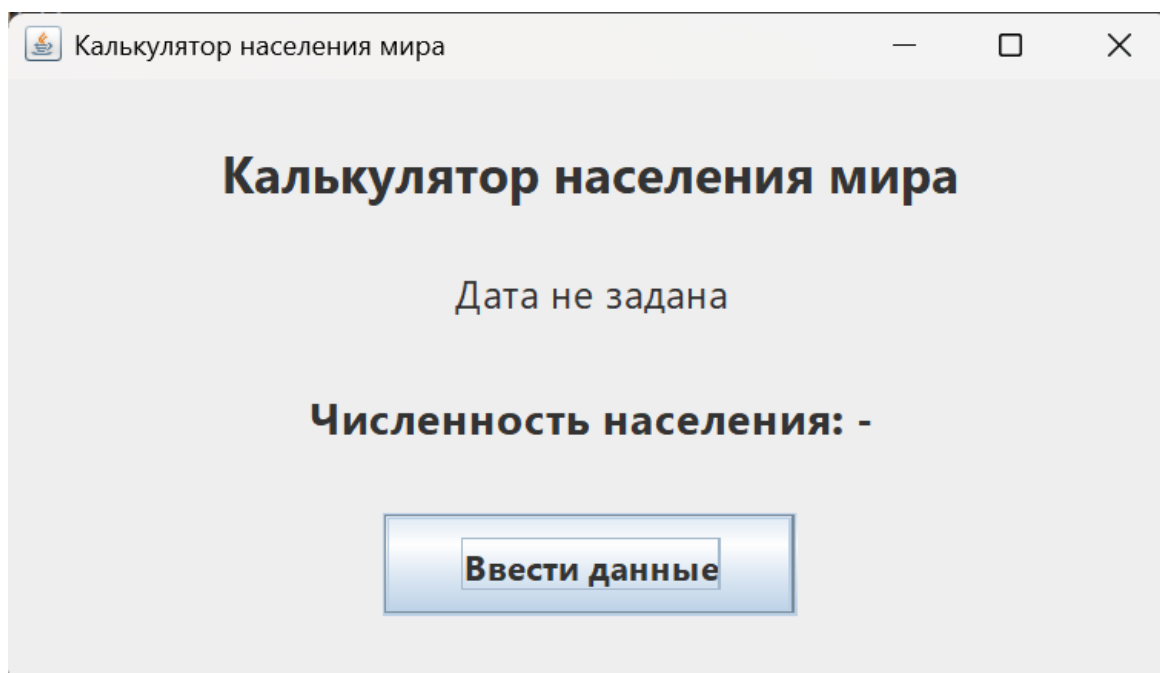


Рисунок 4.1 – Главная форма после запуска

The screenshot shows a dialog box titled 'Ввод даты' (Date Input). The dialog box has a light gray background and a close button (X) in the top right corner. It contains three input fields for date entry. The first field is labeled 'День (1-31):' (Day (1-31):), the second is labeled 'Месяц (1-12):' (Month (1-12):), and the third is labeled 'Год (1900-2200):' (Year (1900-2200):). Each label is in a bold black font, and each input field is a white rectangle with a blue border. At the bottom of the dialog box, there are two blue buttons with white borders: 'Отмена' (Cancel) on the left and 'ОК' (OK) on the right.

Рисунок 4.2 – Окно ввода данных

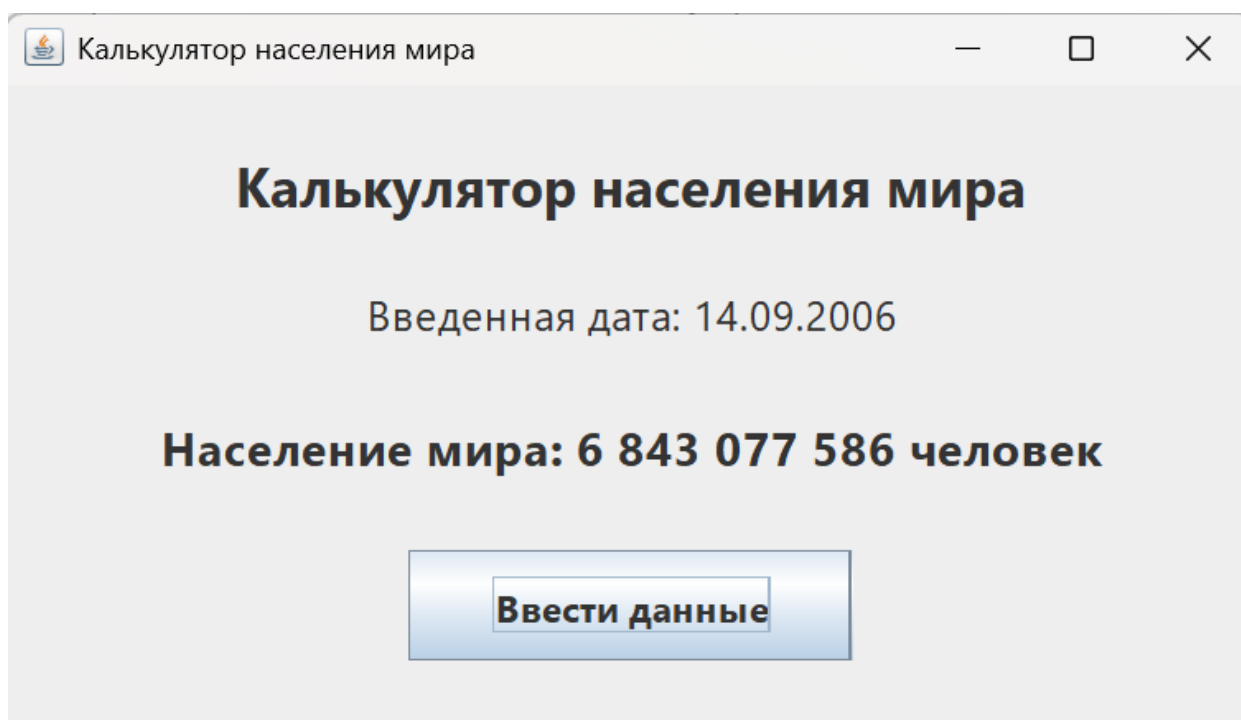


Рисунок 4.3 – Результат вычисления

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1 Какие компоненты входят в состав IDE?

В состав IDE входят редактор кода, компилятор или интерпретатор, система сборки и запуска, отладчик, средства навигации и рефакторинга, терминал, интеграция с системой контроля версий и механизм расширений

Вопрос 2 Каковы основные приёмы работы с редактором кода?

К основным приёмам относятся автодополнение, быстрые исправления, автоматический импорт, форматирование, переход к объявлению, поиск использований, переименование, просмотр иерархии классов и анализ предупреждений.

Вопрос 3 Каким образом выполняется сборка и запуск проекта?

IntelliJ IDEA компилирует Java-файлы командой Build Project. Для запуска выбирается класс App с методом `public static void main(String[] args)` и выполняется команда Run. При запуске в режиме Debug программа работает под управлением отладчика и останавливается на заданных точках. Проект также можно собрать командой `javac --release 17` и запустить командой `java` с указанием класса `com.example.Main`.

Вопрос 4 Что такое MVC?

MVC – архитектурный паттерн, который разделяет данные и предметную логику, графический интерфейс и обработку действий пользователя. В данной работе роль модели выполняет `PopulationModel`, роль представления – `MainView` и `InputDialog`, а роль контроллера – `PopulationController`.

Вопрос 5 Чем отличается активная модель от пассивной?

Пассивная модель не уведомляет представление об изменениях, поэтому контроллер сам обновляет интерфейс, а активная модель сама публикует событие при изменении состояния. В приложении реализована активная модель: `PopulationModel` содержит интерфейс `ModelListener` и список слушателей, после изменения данных метод `setData()` вызывает `notifyListeners()`, оповещая подписчиков, а `MainView` реализует `ModelListener`, подписывается через `model.addListener(this)` и в методе `onModelChanged()` обновляет метки, читая данные через геттеры модели, при этом контроллер в обновлении интерфейса не участвует.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены основные возможности среды IntelliJ IDEA: редактирование исходного кода, сборка проекта, запуск и отладка Java-приложения с графическим интерфейсом. Освоены средства работы с UML-диаграммами — применён плагин PlantUML Integration, диаграмма классов приложения сохранена в файле docs/diagram.puml. Для описания структуры системы использована текстовая нотация SysML v2.

Для варианта 7 разработана утилита расчёта численности населения мира по введённой дате. Программа реализована на языке Java с использованием библиотеки Swing; для ввода даты, согласно заданию, применены текстовые поля, а не стандартный элемент выбора даты. Расчёт основан на экспоненциальной модели роста населения с базовой точкой в январе 2022 года (7,95 млрд человек) и среднегодовым темпом прироста 0,98%; допустимый диапазон вводимых лет — 1900–2200.

Приложение построено по архитектурному паттерну MVC: модель (PopulationModel) хранит состояние и выполняет вычисления, представление (MainView, InputDialog) формирует интерфейс, контроллер (PopulationController) связывает пользователя с моделью. Реализована активная модель — PopulationModel уведомляет подписанное представление через интерфейс ModelListener, и MainView автоматически обновляет отображаемые значения при изменении данных. Реализованы восстановление последних корректных данных и обработка ошибок ввода.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг исходного кода приложения

InputDialog.java

```
package com.example;

import javax.swing.*;
import java.awt.*;

public class InputDialog extends JDialog {

    private final JTextField txtDay = new JTextField();
    private final JTextField txtMonth = new JTextField();
    private final JTextField txtYear = new JTextField();

    public InputDialog(JFrame parent, PopulationController controller) {
        super(parent, "Ввод даты", true);

        setSize(360, 230);
        setResizable(false);

        JPanel panel = new JPanel(new GridLayout(4, 2, 10, 10));
        panel.setBorder(
            BorderFactory.createEmptyBorder(20, 20, 20, 20)
        );

        panel.add(new JLabel("День (1-31):"));
        panel.add(txtDay);

        panel.add(new JLabel("Месяц (1-12):"));
        panel.add(txtMonth);

        panel.add(new JLabel("Год (1900-2200):"));
        panel.add(txtYear);

        JButton btnOk = new JButton("OK");
        JButton btnCancel = new JButton("Отмена");

        panel.add(btnCancel);
        panel.add(btnOk);

        add(panel);

        btnOk.addActionListener(e ->
            controller.processInput(
                txtDay.getText(),
                txtMonth.getText(),
```

```

        txtYear.getText()
    )
);

btnCancel.addActionListener(e -> dispose());

txtDay.addActionListener(e -> btnOk.doClick());
txtMonth.addActionListener(e -> btnOk.doClick());
txtYear.addActionListener(e -> btnOk.doClick());
}

public void setValues(int day, int month, int year) {
    if (year != 0) {
        txtDay.setText(String.valueOf(day));
        txtMonth.setText(String.valueOf(month));
        txtYear.setText(String.valueOf(year));
    } else {
        txtDay.setText("");
        txtMonth.setText("");
        txtYear.setText("");
    }
}
}

```

PopulationController.java

```

package com.example;

import javax.swing.JFrame;
import javax.swing.JOptionPane;

public class PopulationController {

    private final PopulationModel model;
    private MainView view;
    private InputDialog inputDialog;

    public PopulationController(PopulationModel model) {
        this.model = model;
    }

    public void setView(MainView view) {
        this.view = view;
    }

    public void openInputDialog(JFrame parent) {
        if (inputDialog == null || !inputDialog.isDisplayable()) {

```

```

        inputDialog = new InputDialog(parent, this);
    }

    inputDialog.setValues(
        model.getDay(),
        model.getMonth(),
        model.getYear()
    );

    inputDialog.setLocationRelativeTo(parent);
    inputDialog.setVisible(true);
}

public void processInput(String dayText, String monthText, String
yearText) {
    try {
        if (dayText.trim().isEmpty()
            || monthText.trim().isEmpty()
            || yearText.trim().isEmpty()) {

            throw new IllegalArgumentException(
                "Все поля должны быть заполнены."
            );
        }

        int day = Integer.parseInt(dayText.trim());
        int month = Integer.parseInt(monthText.trim());
        int year = Integer.parseInt(yearText.trim());

        model.setData(day, month, year);

        if (inputDialog != null) {
            inputDialog.dispose();
        }

    } catch (NumberFormatException e) {
        showError("День, месяц и год должны быть целыми числами.");
    } catch (IllegalArgumentException e) {
        showError(e.getMessage());
    }
}

private void showError(String message) {
    JOptionPane.showMessageDialog(
        view,
        message,
        "Ошибка ввода",
        JOptionPane.ERROR_MESSAGE
    );
}

```

```

    );
}
}

```

MainView.java

```

package com.example;

import javax.swing.*.*;
import java.awt.*.*;

public class MainView extends JFrame
    implements PopulationModel.ModelListener {

    private final PopulationController controller;
    private final PopulationModel model;

    private final JLabel lblDate =
        new JLabel("Дата не задана", SwingConstants.CENTER);

    private final JLabel lblPopulation =
        new JLabel("Численность населения: -", SwingConstants.CENTER);

    public MainView(
        PopulationController controller,
        PopulationModel model
    ) {
        this.controller = controller;
        this.model = model;

        model.addListener(this);

        initView();
    }

    private void initView() {
        setTitle("Калькулятор населения мира");
        setSize(520, 300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        JPanel mainPanel = new JPanel(new BorderLayout(15, 15));
        mainPanel.setBorder(
            BorderFactory.createEmptyBorder(25, 30, 25, 30)
        );

        JLabel title = new JLabel(
            "Калькулятор населения мира",
            SwingConstants.CENTER
        );
        title.setFont(new Font("Segoe UI", Font.BOLD, 22));

        JPanel resultPanel = new JPanel(
            new GridLayout(2, 1, 10, 10)
        );

        lblDate.setFont(
            new Font("Segoe UI", Font.PLAIN, 17)
        );

        lblPopulation.setFont(

```

```

        new Font("Segoe UI", Font.BOLD, 19)
    );

    resultPanel.add(lblDate);
    resultPanel.add(lblPopulation);

    JButton btnInput = new JButton("Ввести данные");
    btnInput.setFont(
        new Font("Segoe UI", Font.BOLD, 15)
    );

    btnInput.setPreferredSize(
        new Dimension(180, 45)
    );

    btnInput.addActionListener(
        e -> controller.openInputDialog(this)
    );

    JPanel buttonPanel = new JPanel();
    buttonPanel.add(btnInput);

    mainPanel.add(title, BorderLayout.NORTH);
    mainPanel.add(resultPanel, BorderLayout.CENTER);
    mainPanel.add(buttonPanel, BorderLayout.SOUTH);

    add(mainPanel);
}

@Override
public void onModelChanged() {
    lblDate.setText(
        String.format(
            "Введенная дата: %02d.%02d.%d",
            model.getDay(),
            model.getMonth(),
            model.getYear()
        )
    );

    lblPopulation.setText(
        String.format(
            "Население мира: %,d человек",
            model.getCalculatedPopulation()
        )
    );
}
}

```

PopulationModel.java

```

package com.example;

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.List;

public class PopulationModel {

    private static final double START_POPULATION = 7_950_000_000.0;

```

```

private static final int START_YEAR = 2022;
private static final double ANNUAL_GROWTH_RATE = 0.0098;

private int day;
private int month;
private int year;
private long calculatedPopulation;

private final List<ModelListener> listeners = new ArrayList<>();

public interface ModelListener {
    void onModelChanged();
}

public void addListener(ModelListener listener) {
    listeners.add(listener);
}

private void notifyListeners() {
    for (ModelListener listener : listeners) {
        listener.onModelChanged();
    }
}

public void setData(int day, int month, int year) {
    if (year < 1900 || year > 2200) {
        throw new IllegalArgumentException(
            "Год должен находиться в диапазоне от 1900 до 2200."
        );
    }

    if (month < 1 || month > 12) {
        throw new IllegalArgumentException(
            "Месяц должен находиться в диапазоне от 1 до 12."
        );
    }

    if (day < 1 || day > 31) {
        throw new IllegalArgumentException(
            "День должен находиться в диапазоне от 1 до 31."
        );
    }

    try {
        LocalDate.of(year, month, day);
    } catch (Exception e) {
        throw new IllegalArgumentException(
            "Указанной даты не существует."
        );
    }

    this.day = day;
    this.month = month;
    this.year = year;

    calculatePopulation();
    notifyListeners();
}

private void calculatePopulation() {
    LocalDate startDate = LocalDate.of(START_YEAR, 1, 1);
    LocalDate targetDate = LocalDate.of(year, month, day);
    long daysBetween = ChronoUnit.DAYS.between(startDate, targetDate);
    double elapsedYears = daysBetween / 365.2425;
}

```

```

        this.calculatedPopulation =
            (long) (START_POPULATION * Math.exp(ANNUAL_GROWTH_RATE *
elapsedYears));
    }

    public int getDay() {
        return day;
    }

    public int getMonth() {
        return month;
    }

    public int getYear() {
        return year;
    }

    public long getCalculatedPopulation() {
        return calculatedPopulation;
    }

    public boolean hasData() {
        return year != 0;
    }
}

```

Main.java

```

package com.example;

import javax.swing.SwingUtilities;

public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            PopulationModel model = new PopulationModel();
            PopulationController controller = new
PopulationController(model);
            MainView view = new MainView(controller, model);

            controller.setView(view);
            view.setVisible(true);
        });
    }
}

```